Medical X-Ray Device Emulation & Modbus/OpenPLC Protocol Auditing La

1. Overview & Lab Scope

This laboratory environment emulates a networked Medical Digital Radiography (X-Ray) Subsystem controlled via Modbus TCP / OpenPLC and paired with a MedPACS X-Ray Image Server & Web GUI.

Rather than conducting offensive exploitation or unauthorized data exfiltration, students use this environment to:
1. Understand how industrial/medical controllers (Modbus TCP / IEC 61131-3 OpenPLC logic) coordinate physical acquisition parameters (‘kVp‘, ‘mAs‘, door safety interlocks) with a clinical image archive.
2. Inspect how legacy Modbus TCP registers store and transmit device telemetry and study references in cleartext without native protocol authentication.
3. Audit the trust boundary between the Modbus controller and the HTTP PACS backend, and verify defensive remediations (‘REQUIRE_PLC_TOKEN‘ and ‘PLC_READ_ONLY_LOCK‘).

---

2. Dataset Provenance (X-Ray Modality Filter)

The image server ingests only X-ray radiographs from the two specified Hugging Face datasets via [‘image-server/fetch_xrays.py‘](image-server/fetch_xrays.py):

1. [HumynLabs/medical-imaging](https://huggingface.co/datasets/HumynLabs/medical-imaging)
   - Curated anonymized chest/skeletal X-ray radiographs paired with radiology reports.
2. [robailleo/medical-imaging-combined](https://huggingface.co/datasets/robailleo/medical-imaging-combined)
   - Multi-modality dataset containing X-ray, CT, MRI, and Ultrasound samples.
   - The ingestion pipeline strictly filters records where ‘modality == "X-ray"‘ (case-insensitive) and discards all CT, MRI, Ultrasound, and Unknown rows.

---

3. Container Architecture (‘docker-compose.yml‘)

The environment consists of three Docker containers connected to an isolated bridge network (‘mednet‘ ? ‘172.28.0.0/24‘):

Container Name	IP Address	Ports	Role
‘medical-image-server‘	‘172.28.0.10‘	‘8080:8080‘	Flask PACS Web GUI & REST API serving curated X-ray images and logging PLC telemetry
‘xray-modbus-controller‘	‘172.28.0.20‘	‘5020:5020‘	Modbus TCP / OpenPLC-compatible X-Ray generator controller (‘pymodbus‘ + ‘xray_openplc.st‘)
‘protocol-audit-client‘	‘172.28.0.30‘	Internal	Protocol diagnostics & security auditing workstation (‘audit_inspector.py‘, ‘tcpdump‘, ‘curl‘)

---

4. Modbus TCP / OpenPLC Register & Coil Map

The X-Ray controller ([‘xray-controller/controller.py‘](xray-controller/controller.py) and OpenPLC Structured Text [‘xray-controller/xray_openplc.st‘](xray-controller/xray_openplc.st)) exposes the following registers on TCP Port 5020:

Coils (Function Codes ‘01‘ Read Coils / ‘05‘ Write Single Coil)

Coil Address	OpenPLC Address	Name	Description
‘0‘	‘%QX0.0‘	‘DoorInterlock‘	‘1‘ = Radiation room door closed (Safe); ‘0‘ = Door open (‘INTERLOCK_OPEN‘ fault)
‘1‘	‘%QX0.1‘	‘ExposureTrigger‘	Momentary trigger (‘1‘) instructing the PLC to stage the selected study on the Image Server

Holding Registers (Function Codes ‘03‘ Read / ‘06‘ Write Single / ‘16‘ Write Multiple)

Register	OpenPLC Address	Name	Default	Description
‘HR 0‘	‘%QW0‘	‘DeviceStatus‘	‘1‘	‘0‘=IDLE, ‘1‘=READY, ‘2‘=EXPOSING, ‘3‘=TRANSFERRING, ‘4‘=INTERLOCK_OPEN
‘HR 1‘	‘%QW1‘	‘TubeKvp‘	‘85‘	X-ray tube peak kilovoltage (‘40?150 kVp‘)
‘HR 2‘	‘%QW2‘	‘TubeMas‘	‘12‘	Tube current-time product (‘1?200 mAs‘)
‘HR 3‘	‘%QW3‘	‘SelectedStudyIdx‘	‘0‘	Index (‘0..N-1‘) of the active X-ray study staged on the Image Server
‘HR 4‘	‘%QW4‘	‘LastSyncHttpCode‘	‘200‘	Last HTTP status returned by ‘POST /api/plc-sync‘
‘HR 5..20‘	‘%QW5..%QW20‘	‘ActiveImagePath‘	ASCII	16-bit ASCII-encoded URI path of the currently staged X-ray image (e.g., ‘/images/XR-HUMYN-001.png‘)

---

5. Standing Up the Environment

From the ‘medical-xray-lab‘ directory, build and start all three containers:

“‘bash
docker compose up --build -d
“‘

Verify that all three containers are running:

“‘bash
docker compose ps
“‘

Open the MedPACS X-Ray Imaging GUI in your browser:
- URL: ‘http://localhost:8080‘

---

6. Student Lab Exercises: Protocol Auditing & Security Assessment

Exercise 1: Inspecting Modbus TCP Telemetry & Staged Study References

Standard Modbus TCP lacks session authentication and encryption. Any host on the same network segment (‘mednet‘) can query holding registers to inspect device parameters and read the ASCII-encoded path of the active X-ray radiograph stored in ‘HR 5..20‘.

Run the included diagnostic inspector inside the ‘protocol-audit-client‘ container:

‘‘‘bash
docker exec -it protocol-audit-client python audit_inspector.py
‘‘‘

What to Observe:
- The script connects to ‘xray-modbus-controller:5020‘ using ‘pymodbus‘ Function Code ‘03‘ (Read Holding Registers) and Function Code ‘01‘ (Read Coils).
- ‘hr_5_20_staged_image_path‘ reveals the exact URI path on the image server (such as ‘/images/XR-HUMYN-001.png‘) corresponding to ‘hr_3_selected_study_index‘.
- Simultaneously, check the Controller & Access Audit Log table in the Web GUI (‘http://localhost:8080‘) to see how requests are logged.

Exercise 2: Analyzing Controller-to-PACS Trust Boundaries

Review how the controller synchronizes state with the Image Server in [‘xray-controller/controller.py‘](xray-controller/controller.py):
1. Every 1.5 seconds, the controller reads ‘HR 1‘ (‘TubeKvp‘), ‘HR 2‘ (‘TubeMas‘), ‘HR 3‘ (‘SelectedStudyIdx‘), and ‘Coil 1‘ (‘ExposureTrigger‘).
2. It sends a JSON payload to ‘POST http://medical-image-server:8080/api/plc-sync‘.
3. In the default baseline configuration (‘REQUIRE_PLC_TOKEN=false‘ and ‘PLC_READ_ONLY_LOCK=false‘), neither the Modbus registers nor the HTTP synchronization endpoint enforce authentication or write protection.

Discussion Questions for Students:
- Why is placing a Modbus TCP controller on the same flat subnet as a PACS image archive dangerous in a hospital network?
- If an unauthorized device modifies ‘HR 1‘ (‘TubeKvp‘), ‘HR 2‘ (‘TubeMas‘), or ‘Coil 0‘ (‘DoorInterlock‘), what are the clinical and patient-safety implications?
- How does encoding file paths or study identifiers inside world-readable PLC registers (‘HR 5..20‘) create an information disclosure risk?

Exercise 3: Applying & Verifying Defensive Hardening

To remediate the identified vulnerabilities, update the environment variables in [‘docker-compose.yml‘](docker-compose.yml):

1. Enable PLC-to-Server Token Authentication on ‘medical-image-server‘:
   ‘‘‘yaml
   environment:
     - REQUIRE_PLC_TOKEN=true
     - PLC_SHARED_TOKEN=medlab-secure-plc-token-2026
   ‘‘‘
2. Enable Register Write Protection Lock on ‘xray-modbus-controller‘:
   ‘‘‘yaml
   environment:

- PLC_READ_ONLY_LOCK=true
- PLC_SHARED_TOKEN=medlab-secure-plc-token-2026

```

3. Recreate the containers:

```bash
docker compose up -d
```

4. Verify in the Web GUI (‘http://localhost:8080‘) that the top-right security badge now displays ‘AUTHENTICATED (HARDENED)‘, and confirm via ‘audit_inspector.py‘ that unauthenticated ‘/api/plc-sync‘ requests without ‘X-PLC-Token‘ are rejected with ‘HTTP 401‘ and logged as ‘PLC_AUTH_REJECTED‘.
```